

KV-Cache Memory Complexity.

At inference an autoregressive transformer would otherwise recompute the keys and values of every past token at every step. Caching them turns $O(T^2)$ recompute into $O(T)$ memory — but that memory is linear in sequence length and quickly dominates. Below we derive the exact byte formula and compute a 7B-class model's cache at 2K, 8K, and 32K tokens, to the byte.

THE EQUATION

$$\text{cache bytes} = 2 \cdot L \cdot T \cdot d_{\text{model}} \cdot b \cdot B$$

Worked example. $L=32$, $d_{\text{model}}=4096$, fp16 ($b=2$), batch $B=1$, $T=2048$: $2 \cdot 32 \cdot 2048 \cdot 4096 \cdot 2 = 1,073,741,824$ bytes = 1.00 GiB $\approx$ 1.07 GB.

Topic 5 of 5 in this module · companion to the course page · v1.0

Contents

1	The claim	2
2	Deriving the $O(LTd)$ size and the $O(T^2) \rightarrow O(T)$ trade	3
3	Worked example: a 7B-class model	3
4	Consequences that matter	4
5	Where this sits in the track	4
A	Reproducible (NumPy)	4

1 The claim

During autoregressive generation, attention at step t needs the keys K and values V of *all* previous tokens $1, \dots, t$. Those past K, V do not change as new tokens arrive, so we compute them once and *cache* them per layer. The price is memory:

$$\text{cache bytes} = \underbrace{2}_{K,V} \cdot L \cdot T \cdot d_{\text{model}} \cdot b \cdot B \quad (1)$$

Symbol	Typical	Meaning
2	—	one tensor each for K and V
L	32	number of transformer layers
T	varies	sequence length (cached tokens)
d_{model}	4096	model width (summed over all heads)
b	2	bytes per element (fp16 = 2, fp32 = 4)
B	1	batch size

Without the cache, generating token 2000 would recompute the key and value of all 1999 earlier tokens — and you would redo that on every single step, an $O(T^2)$ avalanche of arithmetic. The cache says: “I already worked out each past token’s K and V once; I’ll keep them on the shelf.” Generation becomes: compute K, V for the *one* new token, append to the shelf, attend. The shelf, however, grows by one slot per token, per layer — and the shelf is what eventually fills the GPU.

2 Deriving the $O(LTd)$ size and the $O(T^2) \rightarrow O(T)$ trade

Per token, per layer. One token's key is a vector in $\mathbb{R}^{d_{\text{model}}}$ and so is its value. Storing both costs $2 d_{\text{model}} b$ bytes. Across L layers that is

$$\text{per token} = 2 L d_{\text{model}} b \text{ bytes} \quad (B = 1).$$

Whole sequence. For T tokens, multiply by T :

$$\text{cache} = 2 L T d_{\text{model}} b B = O(L T d_{\text{model}}),$$

recovering (1): *linear* in sequence length.

The trade. Each generated token grows the cache by $O(d_{\text{model}})$ per layer, so total memory is $O(LTd)$. The alternative — recomputing all past K, V at each of T steps — costs $O(T^2)$ arithmetic. The cache buys an $O(T^2) \rightarrow O(T)$ *compute* reduction at the cost of $O(T)$ *memory*:

$$\boxed{\text{recompute } O(T^2) \text{ FLOPs} \rightarrow \text{cache } O(T) \text{ memory}} \quad (2)$$

3 Worked example: a 7B-class model

Take $L = 32$, $d_{\text{model}} = 4096$, fp16 so $b = 2$ bytes, batch $B = 1$.

At $T = 2048$:

$$2 \cdot 32 \cdot 2048 \cdot 4096 \cdot 2 = 1,073,741,824 \text{ bytes} = \frac{1,073,741,824}{1024^3} \text{ GiB} = 1.00 \text{ GiB} \approx 1.07 \text{ GB}.$$

Building it up: $2 \cdot 32 = 64$; $64 \cdot 2048 = 131,072$; $131,072 \cdot 4096 = 536,870,912$; $\times 2 = 1,073,741,824$ bytes. (Per token, all layers: $2 \cdot 32 \cdot 4096 \cdot 2 = 524,288$ bytes = 0.5 MiB.)

At $T = 8192$ ($4\times$): 4,294,967,296 bytes = 4.00 GiB $\approx$ 4.29 GB.

At $T = 32768$ ($16\times$): 17,179,869,184 bytes = 16.00 GiB $\approx$ 17.18 GB.

T	bytes	GiB	GB (10^9)
2048	1,073,741,824	1.00	1.07
8192	4,294,967,296	4.00	4.29
32768	17,179,869,184	16.00	17.18

The growth is exactly linear in T : heatmap of cache size (intensity scales with GiB, cap 80):

	$T=2048$	$T=8192$	$T=32768$
cache	1.00 GiB	4.00 GiB	16.00 GiB

At 32K tokens the KV-cache alone is 16 GiB — comparable to the model weights themselves and enough to dominate the memory budget of a single GPU.

The KV-cache scales as $2LTd_{\text{model}}bB$: *linear* in every factor, but T is the one that grows during generation. Doubling the context window doubles the cache; a $16\times$ longer context is a $16\times$ larger cache. This linear-in- T memory — not compute — is the binding constraint on long-context inference.

4 Consequences that matter

1. **Memory, not FLOPs, caps context.** Per-token decode compute is cheap; the cache is what runs you out of VRAM.
2. **Batch multiplies it.** B concurrent sequences each need their own cache; serving throughput trades directly against context length.
3. **Precision halves or doubles it.** fp16/bf16 ($b=2$) is standard; fp8 KV ($b=1$) halves the cache, fp32 ($b=4$) doubles it.
4. **Heads are folded into d_{model} .** Since $d_{\text{model}} = h \cdot d_{\text{head}}$, the formula already sums over all h attention heads — which is exactly what GQA attacks next.

The leading 2 is for K and V — not a fudge factor and not optional. Two more common slips: (i) forgetting to multiply by L (the cache is per layer, all layers live simultaneously); (ii) confusing GiB (1024^3) with GB (10^9) — our $T=2048$ cache is exactly 1.00 GiB but 1.07 GB. State the unit. And note this counts *only* the cache; model weights and activations are on top.

5 Where this sits in the track

Link	Connection
Module 0.1	The Q, K, V projections whose K, V are what we cache.
Module 3.1	GQA (grouped-query attention): share K, V across groups of heads, shrinking the effective d_{model} in the cache by the group factor.
Module 7.3	PagedAttention : manage the cache in fixed-size pages (vLLM) to eliminate fragmentation and overcommit memory across requests.

A Reproducible (NumPy)

Inputs: $L = 32$, $d_{\text{model}} = 4096$, $b = 2$ (fp16), $B = 1$. Outputs: $T=2048 \rightarrow 1,073,741,824$ B (1.00 GiB); $T=8192 \rightarrow 4,294,967,296$ B (4.00 GiB); $T=32768 \rightarrow 17,179,869,184$ B (16.00 GiB).

```
import numpy as np
L, d, b, B = 32, 4096, 2, 1
cache = lambda T: 2 * L * T * d * b * B           # bytes
```

```
for T in (2048, 8192, 32768):
    by = cache(T)
    print(T, by, by/1024**3, "GiB", by/1e9, "GB")
# 2048    1073741824    1.0 GiB    1.073741824 GB
# 8192    4294967296    4.0 GiB    4.294967296 GB
# 32768   17179869184   16.0 GiB   17.179869184 GB
print("per token, all layers:", 2*L*d*b, "bytes")    # 524288 (0.5 MiB)
```

Marevlo Research · Transformer Track · Module 0.2 · Topic 5 of 5.